

Cloud Computing TA-1

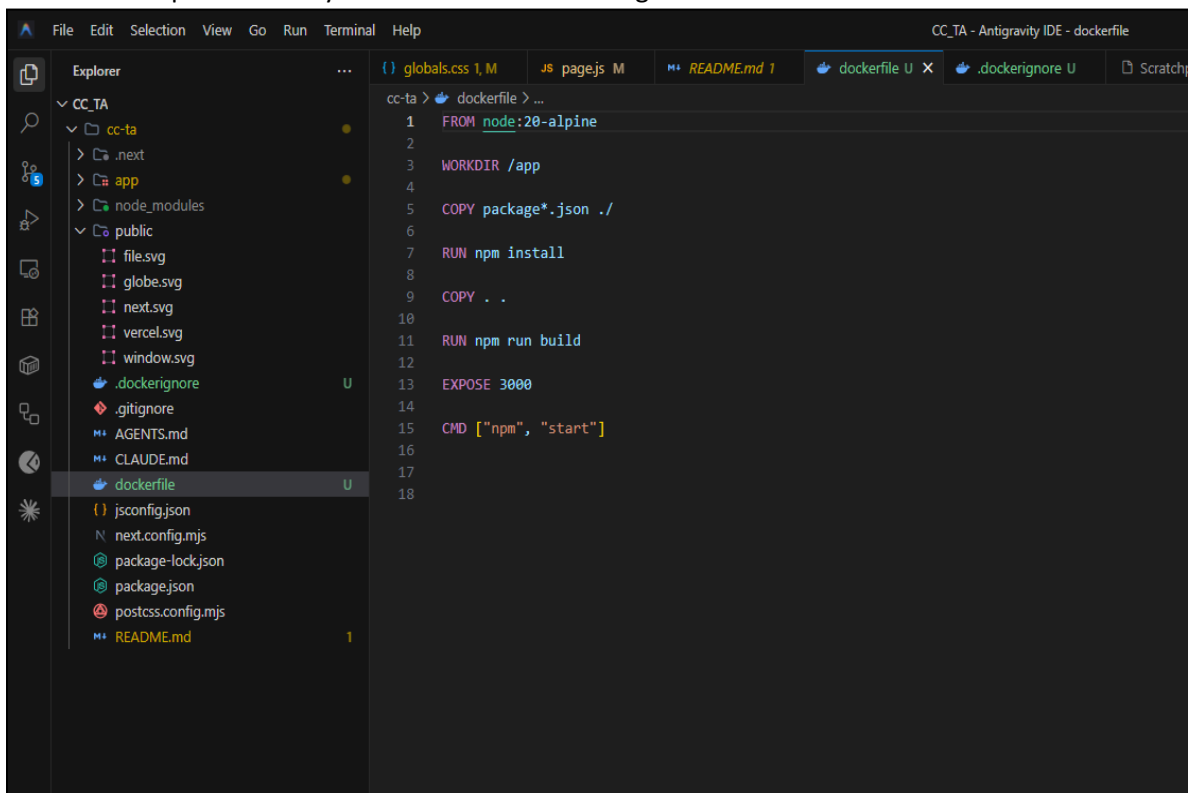
Dockerization and AWS Deployment of a Web Application

Name	Rohit Agrawal
Section / Batch	A4 / B3
Roll No	46
Subject	Cloud Computing

Topic: Create and implement a Dockerfile of a self-created web application and host it on AWS.

1. Writing the Dockerfile

The Dockerfile was created for the Next.js web application. It uses node:20-alpine as the base image, sets up the working directory at /app, copies the dependency files (package.json and package-lock.json), installs packages using npm install, copies the remaining source code, builds the production bundle with npm run build, exposes port 3000, and defines the startup command as npm start. Each instruction in the Dockerfile corresponds to a layer in the final Docker image.

A screenshot of an IDE window titled "CC_TA - Antigravity IDE - dockerfile". The left sidebar shows a file explorer with a project structure: "CC_TA" containing ".next", "app", "node_modules", and "public". The "public" directory contains "file.svg", "globe.svg", "next.svg", "vercel.svg", and "window.svg". There are also ".dockerignore", ".gitignore", "AGENTS.md", "CLAUDE.md", "dockerfile", "jsconfig.json", "next.config.mjs", "package-lock.json", "package.json", "postcss.config.mjs", and "README.md". The main editor area shows the content of the "dockerfile" file, which contains the following instructions:

```
1 FROM node:20-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN npm install
8
9 COPY . .
10
11 RUN npm run build
12
13 EXPOSE 3000
14
15 CMD ["npm", "start"]
16
17
18
```

2. Building the Docker Image

The command "docker build -t rohit_cc_ta_1 ." was executed to build the Docker image. Docker processed all 6 steps defined in the Dockerfile: pulling the Node.js 20 Alpine base image, setting the working directory, copying package files, running npm install (15.1s), copying source code, and running the production build (6.7s). The total build time was 64.9 seconds. The final image was tagged as rohit_cc_ta_1:latest.

```
PS D:\CC_TA\cc-ta> docker build -t rohit_cc_ta_1 .
>>
[+] Building 64.9s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from dockerfile                0.0s
=> => transferring dockerfile: 196B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  1.8s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 64B                                      0.0s
=> [1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a8b0d56d50c58a04632e6c0fb8363f609372293  6.6s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a8b0d56d50c58a04632e6c0fb8363f609372293  0.0s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B 0.3s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB 0.7s
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB 5.7s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 1.2s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.1s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 0.8s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 0.0s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 885B                                      0.0s
=> [2/6] WORKDIR /app                                              0.1s
=> [3/6] COPY package*.json ./                                     0.0s
=> [4/6] RUN npm install                                           15.1s
=> [5/6] COPY . .                                                  0.1s
=> [6/6] RUN npm run build                                         6.7s
=> exporting to image                                              34.1s
=> => exporting layers                                              29.8s
=> => exporting manifest sha256:9d8b7dc0b1c0edc39cb81acb2291cbc273ad9b878dac614a34d6c1da08c496af 0.0s
=> => exporting config sha256:40dae4f28d36b8e75b5783ffdcda4664f66cf75073a7201c6318e6839bb5388f 0.0s
=> => exporting attestation manifest sha256:88c45f5fe32951f4441932d8535b33cf1855bd37f100086e73ab3f4010b84c2e 0.1s
=> => exporting manifest list sha256:2aaea10155f0f76d15ec074baa26b19405760d4314f9892f5c6b02f2e0de8316 0.0s
=> => naming to docker.io/library/rohit_cc_ta_1:latest             0.0s
=> => unpacking to docker.io/library/rohit_cc_ta_1:latest          4.0s
```

3. Running the Docker Container Locally

The command "docker run -p 3000:3000 rohit_cc_ta_1" was executed to start the container. The -p 3000:3000 flag maps port 3000 on the host machine to port 3000 inside the container. The Next.js 16.3.0 production server started and became ready in 140ms, accessible at <http://localhost:3000>.

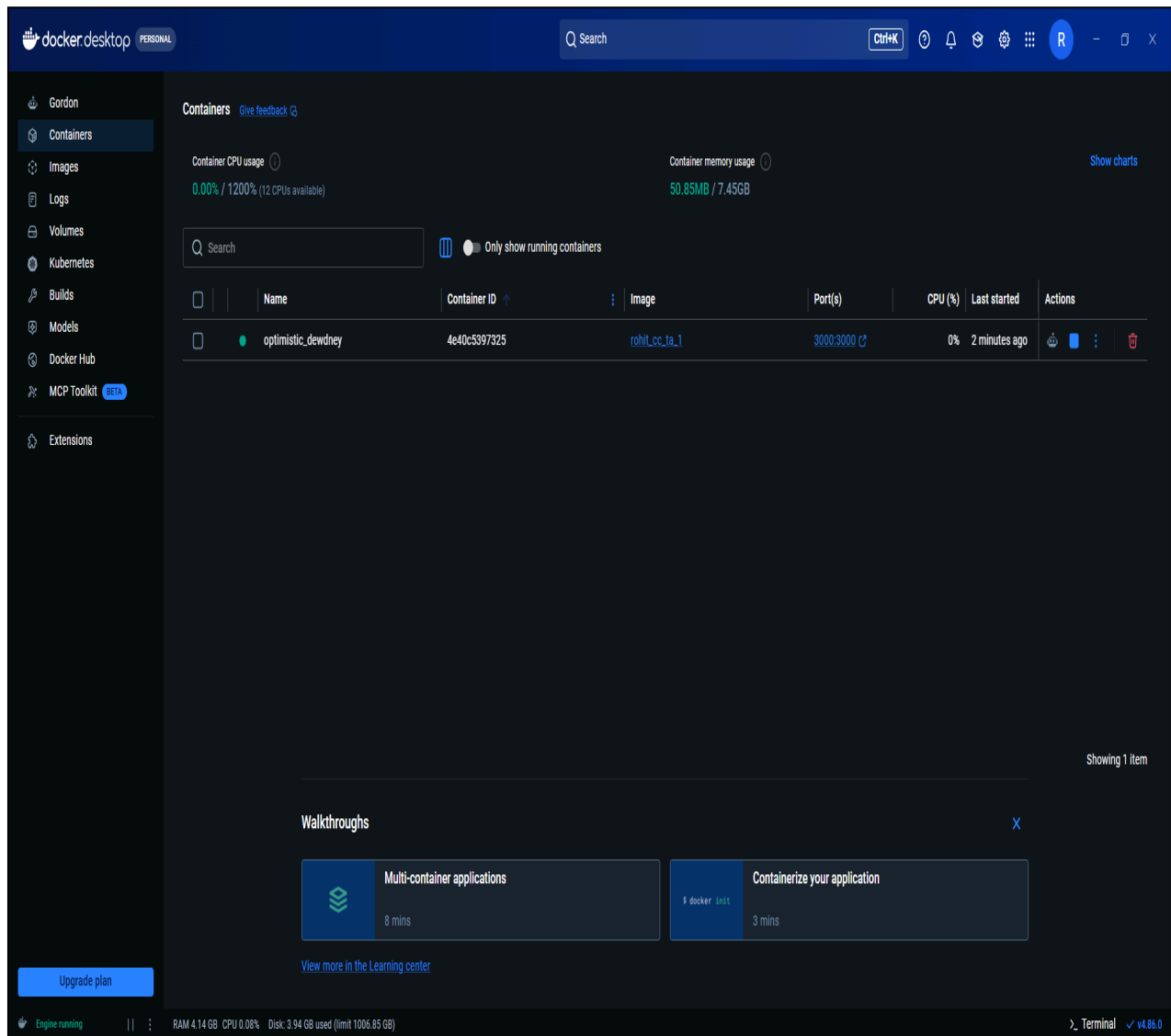
```
PS D:\CC_TA\cc-ta> docker run -p 3000:3000 rohit_cc_ta_1
>>

> cc-ta@0.1.0 start
> next start

▲ Next.js 16.3.0
- Local:      http://localhost:3000
- Network:    http://172.17.0.2:3000
✓ Ready in 140ms
✓ Running next.config.mjs took 24ms
█
```

4. Verifying the Container in Docker Desktop

Docker Desktop shows the running container with the name `optimistic_dewdney`, container ID `4e40c5397325`, using the `rohit_cc_ta_1` image. The container is mapped to port `3000:3000` and has been running for 2 minutes. CPU usage is at 0% and memory consumption is 50.85MB out of 7.45GB available.



5. Tagging the Image for Docker Hub

The command "docker tag rohit_cc_ta_1 rohit1911/rohit_cc_ta_1" was executed to create a new tag for the image that includes the Docker Hub username rohit1911. This is required because Docker Hub expects images in the username/image_name format for pushing to a remote repository.

```
error parsing reference: 'rohit1911/rohit_cc_ta_1' is not a valid repository/registry address
PS D:\CC_TA\cc-ta> docker tag rohit_cc_ta_1 rohit1911/rohit_cc_ta_1
>>
```

6. Pushing the Image to Docker Hub

The command "docker push rohit1911/rohit_cc_ta_1" was executed to upload the image to Docker Hub. All 12 layers of the image were pushed to the repository at docker.io/rohit1911/rohit_cc_ta_1. The final digest confirms the image was uploaded successfully with a size of 856 bytes for the manifest.

```
PS D:\CC_TA\cc-ta> docker push rohit1911/rohit_cc_ta_1
>>
Using default tag: latest
The push refers to repository [docker.io/rohit1911/rohit_cc_ta_1]
74cf79449e47: Pushed
44136fa355b3: Pushed
4feea04c1543: Pushed
6a0ac1617861: Pushed
b2cbbfe903b0: Pushed
fff4e2c1b189: Pushed
69e46583026c: Pushed
a3eccdf7113c: Pushed
991323cc8e4b: Pushed
7ae3832a05d4: Pushed
fda45c4835d8: Pushed
latest: digest: sha256:2aaaa10155f0f76d15ec074baa26b19405760d4314f9892f5c6b02f2e0de8316 size: 856
PS D:\CC_TA\cc-ta> 
```

7. Launching an EC2 Instance on AWS

An EC2 instance was configured on the AWS console with the name CC-TA-1. The Amazon Machine Image (AMI) selected was Amazon Linux 2023 (kernel-6.18), which is a free tier eligible, general-purpose Linux-based operating system optimized for AWS. The 64-bit (x86) architecture was chosen.

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

CC-TA-1

[Add additional tags](#)

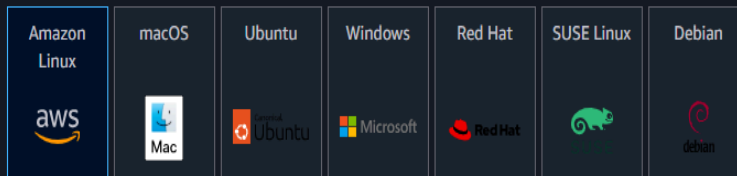
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents

Quick Start



[Browse more AMIs](#)

Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.18 AMI
ami-035827357e3c7e810 (64-bit (x86), uefi-preferred) / ami-00b0a08d4568c22e8 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

Description

Amazon Linux 2023 (kernel-6.18) is a modern, general purpose Linux-based OS that will be supported until June 2029. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.12.20260803.3 x86_64 HVM kernel-6.18

8. Configuring Security Group Rules

A security group rule was added to allow inbound traffic on port 3000 using Custom TCP protocol. The source type was set to Anywhere (0.0.0.0/0) to allow public access to the web application from any IP address. This is necessary for the application to be accessible over the internet.

▼ Security group rule 4 (TCP, 3000, 0.0.0.0/0) Remove

Type Info Custom TCP ▼	Protocol Info TCP
Port range Info 3000	Source type Info Anywhere ▼
Source Info Add CIDR, prefix list or security group 0.0.0.0/0 ✕	Description - optional Info e.g. SSH for admin desktop

9. EC2 Instance Running

The EC2 instance CC-TA-1 was launched with instance ID i-095da9837c11fc4ba. The instance state shows Running with a t3.micro instance type in the ap-south-1 (Mumbai) availability zone. The status check was initializing at the time of the screenshot.

Instances (1) Info							less than
Saved filter sets Choose filter set ▼		Find Instance by attribute or tag (case-sensitive)					
☐	Name 🔗 ▼	Instance ID	Instance state ▼	Instance type ▼	Status check	Availab	
☐	CC-TA-1	i-095da9837c11fc4ba	Running 🔍 🔍	t3.micro	Initializing	ap-sout	

10. SSH Connection to the EC2 Instance

The command "ssh -i CC_TA.pem ec2-user@13.206.84.82" was executed from the local machine to establish an SSH connection to the EC2 instance. The connection was made using the private key file CC_TA.pem. The terminal shows the Amazon Linux 2023 welcome banner, confirming the SSH session was established at ec2-user@ip-172-31-8-14.

```
PS C:\Users\Rahul\Downloads> ssh -i "CC_TA.pem" ec2-user@13.206.84.82

#_
~_  #####_      Amazon Linux 2023
NN_ \#####\
NN_ \###|
NN_ \#/
NN_ V~' '->      https://aws.amazon.com/linux/amazon-linux-2023
NNN
NN_ .-.-
NN_ -/_-/_-/_-
NN_ -/m/'

[ec2-user@ip-172-31-8-14 ~]$
```

11. Installing Docker on the EC2 Instance

The command "sudo yum install docker -y" was executed on the EC2 instance to install Docker. The package manager resolved dependencies and installed Docker version 25.0.16 along with 10 additional dependency packages including containerd, iptables-libs, libcgroup, and runc. The total installation comprised 11 packages.

```
Complete!
[ec2-user@ip-172-31-8-14 ~]$ sudo yum install docker -y
Last metadata expiration check: 0:02:02 ago on Mon Aug 10 14:02:03 2026.
Dependencies resolved.

=====
Package                        Architecture Version                        Repository                    Size
=====
Installing:
docker                         x86_64      25.0.16-1.amzn2023.0.3      amazonlinux                   47 M
Installing dependencies:
container-selinux              noarch      4:2.245.0-1.amzn2023        amazonlinux                   58 k
containerd                     x86_64      2.2.5-1.amzn2023.0.1        amazonlinux                   24 M
iptables-libs                  x86_64      1.8.8-3.amzn2023.0.2        amazonlinux                   401 k
iptables-nft                   x86_64      1.8.8-3.amzn2023.0.2        amazonlinux                   183 k
libcgroup                      x86_64      3.0-1.amzn2023.0.1          amazonlinux                   75 k
libnetfilter_conntrack         x86_64      1.0.8-2.amzn2023.0.2        amazonlinux                   58 k
libnftnl                       x86_64      1.0.1-19.amzn2023.0.2       amazonlinux                   30 k
libnftnl                       x86_64      1.2.2-2.amzn2023.0.2       amazonlinux                   84 k
pigz                           x86_64      2.5-1.amzn2023.0.4          amazonlinux                   83 k
runc                           x86_64      1.3.5-1.amzn2023.0.2        amazonlinux                   3.9 M

Transaction Summary
=====
Install 11 Packages
```

12. Starting the Docker Service on EC2

The command "sudo service docker start" was executed to start the Docker daemon on the EC2 instance. The system redirected the request to systemctl, which started the docker.service. No errors were reported, confirming Docker was successfully started and ready to manage containers.


```
[ec2-user@ip-172-31-8-14 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-8-14 ~]$
```

13. Pulling the Docker Image on EC2

The command "sudo docker pull rohit1911/rohit_cc_ta_1" was executed on the EC2 instance to download the Docker image from Docker Hub. All 12 layers were pulled and the image was downloaded with the digest matching the one from the push step. The status confirms the latest version of the image was downloaded from docker.io/rohit1911/rohit_cc_ta_1.

```
[ec2-user@ip-172-31-8-14 ~]$ sudo docker pull rohit1911/rohit_cc_ta_1
Using default tag: latest
latest: Pulling from rohit1911/rohit_cc_ta_1
6a0ac1617861: Pull complete
4feeaa04c1543: Pull complete
b2cbbfe903b0: Pull complete
fff4e2c1b189: Pull complete
69e46583026c: Pull complete
a3eccdf7113c: Pull complete
991323cc8e4b: Pull complete
7ae3832a05d4: Pull complete
fda45c4835d8: Pull complete
Digest: sha256:2aaea10155f0f76d15ec074baa26b19405760d4314f9892f5c6b02f2e0de8316
Status: Downloaded newer image for rohit1911/rohit_cc_ta_1:latest
docker.io/rohit1911/rohit_cc_ta_1:latest
```

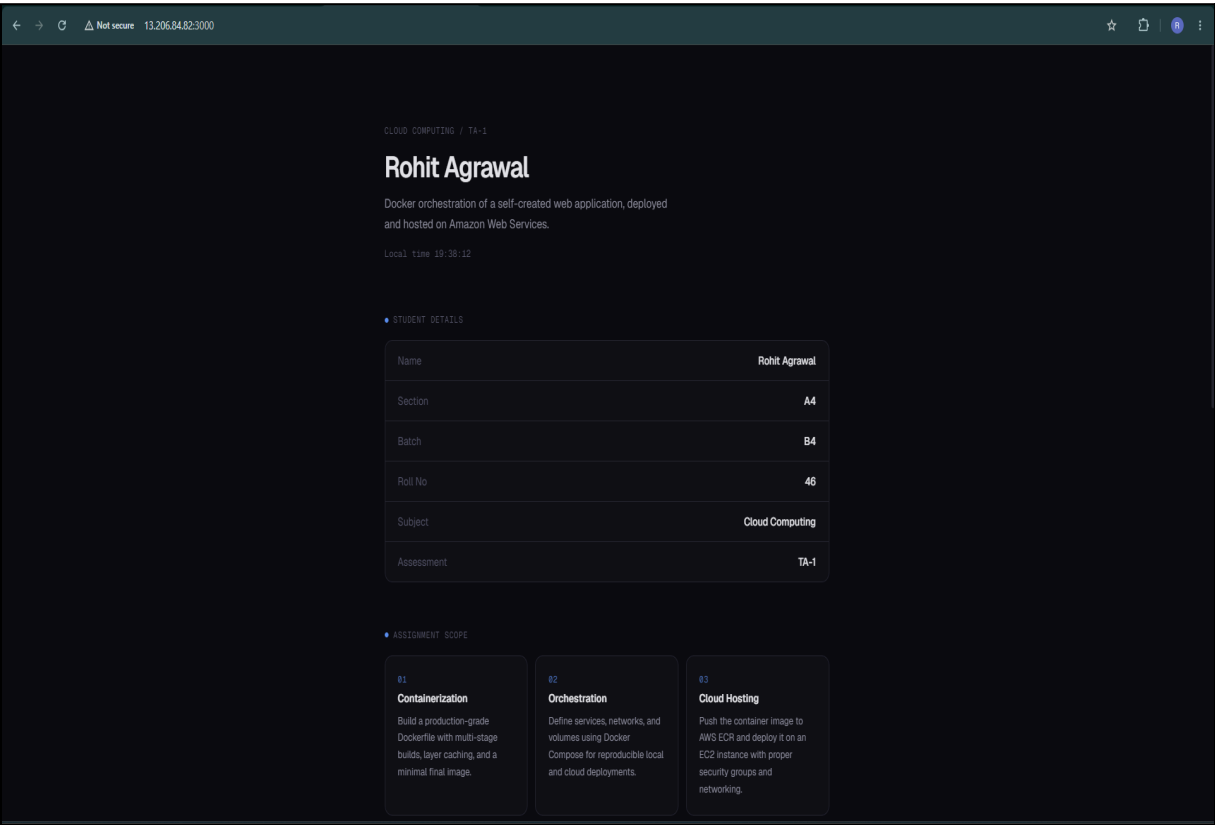
14. Running the Container on EC2

The command "sudo docker run -d -p 3000:3000 rohit1911/rohit_cc_ta_1" was executed to start the container in detached mode (-d) on the EC2 instance. The container ID was returned, confirming the container started in the background with port 3000 mapped from host to container.

```
[ec2-user@ip-172-31-8-14 ~]$ sudo docker run -d -p 3000:3000 rohit1911/rohit_cc_ta_1
64b9d464164dd18ebca2853cc09a96125f3c54ad428d20f7248ad0517a49469b
[ec2-user@ip-172-31-8-14 ~]$
```

15. Web Application Live on AWS

The web application is accessible at <http://13.206.84.82:3000> via the EC2 instance public IP address. The page displays the student information (Rohit Agrawal, Section A4, Batch B4, Roll No 46, Cloud Computing, TA-1), the assignment scope covering Containerization, Orchestration, and Cloud Hosting, along with a live local time display. The application is fully functional and hosted on AWS.



DOCKER FILE

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

EXPOSE 3000

CMD ["npm", "start"]
```

Submitted by: Rohit Agrawal | Section A4 | Batch B3 | Roll No 46